

Univerzita Karlova
Přírodovědecká fakulta



Kryštof Žďárský

III. ročník, B-GEKA

Úvod do programování

Dokumentace k případům ke zkoušce

Předmět „Úvod do programování“

Akademický rok 2025/2026

Jičín, 4.2. 2026

Zadání

1. *Vizualizace Kochovy vločky.*

S využitím želví grafiky vykreslete Kochovu vločku. Vstupní parametry představují délka strany vločky a její stupeň. Hloubku rekurze nastavte maximálně na 5.

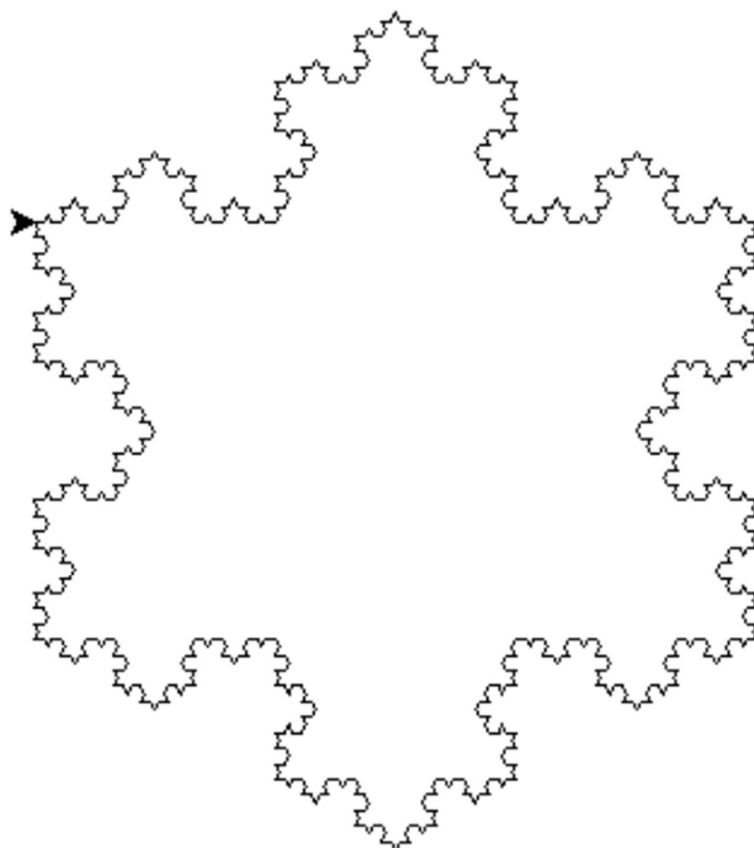
2. *Převod textu do Morseovy abecedy a zpět.*

Vstupní text zahrnující písmena "A-Z", číslovky "0-9", speciální znaky ".,?!;" oddělené mezerami, načtený z textového souboru, převedte do Morseovy abecedy a zpět. Pokud budou v textu nepodporované znaky, ignorujte je. Zkonvertovaný text uložte do souboru.

Vizualizace Kochovy vločky

Cílem této úlohy bylo vytvořit kód, který by dle zadání uživatele vykreslil Kochovu vločku. Hlavním principem Kochovy vločky je opakování Kochovy posloupnosti. Princip konstrukce spočívá v rekurzivním procesu. Pro stupeň $n > 0$ se úsečka nahradí čtyřmi úsečkami o třetinové délce s úhly natočení 0° , 60° , -120° a 60° . Pro $n = 0$ se vykreslí přímá úsečka.

Tato úloha se dá řešit více způsoby, a to i bez využití OOP, ale dle zadání jsem OOP měl způsoby, proto je úloha řešena tímto způsobem.



Obr. 1 – vykreslená Kochova Vločka 4 stupně

Popis kódu

Celý program jsem zapouzdřil do třídy *KochFlake* jako první funkce je inicializační konstruktor `__init__` přebírá vstupní parametry. Z důvodu potenciální náročnosti vykreslování a limitů v zadání obsahuje podmínku, která omezuje maximální stupeň rekurze na 5. Podmínka zároveň obsahuje kontrolu uživatelského vstupu, která při zadání vyšší hodnoty hodnotu změni na hodnotu 5 a při nižší hodnotě zadá hodnotu 1.

Další část programu tvoří metoda *koch_order*, která je hlavním principem Kochovy vložky. Pokud je zadán stupeň rekurze 0 vykreslí se pouze rovná čára o délce d . pokud je rekurze $n > 0$ rozdělí se křivka na 4 Kochovy křivky řádu $n-1$ a o délce $d/3$. Mezi těmito jednotlivými rekurzivními voláními se mění směr kreslení dle vzoru. První křivka otočit o 60° vlevo, druhá křivka otočit o 120° vpravo, třetí křivka otočit o 60° vlevo a čtvrtá křivka rovně. Matematický princip Kochovy křivky, úsečka definována body A a B. V každém kroku iterace je tato úsečka rozdělena na tři shodné části body C a D, kde platí vektorová rovnost:

$$C = A + \frac{1}{3}(B - A) \quad D = A + \frac{2}{3}(B - A)$$

Nad prostřední úsečkou CD je sestaven rovnostranný trojúhelník s vrcholem E . Souřadnice vrcholu E lze získat rotací bodu D kolem bodu C o úhel 60° (nebo $-\pi/3$ radiánů).

```
Funkce koch_order(délka, řád):
    Vstup: délka (reálné číslo), řád (celé číslo)

    1. Base Case (Podmínka ukončení):
        POKUD řád == 0:
            Posuň želvu vpřed o vzdálenost 'délka'
            NÁVRAT

    2. Rekurzivní krok:
        nová_délka = délka / 3

        Zavolej koch_order(new_length, řád - 1)
        Otoč želvu vlevo o 60°
        Zavolej koch_order(new_length, řád - 1)
        Otoč želvu vpravo o 120°
        Zavolej koch_order(new_length, řád - 1)
        Otoč želvu vlevo o 60°
        Zavolej koch_order(new_length, řád - 1)
```

Obr. 2 – Algoritmus kochovy posloupnosti

Třetí část třídy tvoří metoda *flake*, která jednoduchým cyklem uzavírá vložku. V rámci této metody je i příprava želvy pro kreslení vložky a to především její vycentrování, aby vložka jako tatová byla vycentrovaná na obrazovce. Tento jednoduchý cyklus pouze 3-krát opakuje Kochovu posloupnost. Po vykreslení první první křivky se želva otočí o 120° a vykreslí druhou křivku a opět se otočí o 120° a po vykreslení třetí křivky je vložka uzavřena a cyklus je ukončen. Vložka je tak kompletní.

Jako poslední část kódu je hlavní program zajišťující interakci s uživatelem. Uživatel zadává 3 vstupy a to délku 1 strany, která ovlivňuje celou velikost vložky, stupeň rekurze ten ovlivňuje jaký stupeň rekurze je použit (jak moc je vložka “zubatá”) a oproti zadání jsem přidal ještě vstup na kontrolu rychlosti vykreslování. Poslední část hlavního programu je inicializace vykreslovacího okna a finální inicializační část želvy grafiky.

Data a problematika zpracování

Vstupní data do programu jsou pouze uživatelské vstupy, které ovlivňují velikost tvar a vykreslování vložky. Hlavním problémem pro mé zpracování bylo správné pochopení principu Kochovy křivky, by Prostor pro vylepšení bych viděl především v možnosti dalších úprav, které může uživatel provádět a to jako měnit barvu nebo například výsledný “orázek” uložit do souborů.

Převod textu do Morseovy abecedy a zpět

Morsesova abeceda přesněji Morseův kód je skupina symbol obecně využívaná v telegrafii. Kóduje znaky latinské abecedy, číslice a speciální znaky do kombinací krátkých a dlouhých signálů. Tyto znaky se na dlouhé vzdálenosti přenáší snadněji než znaky standartní abecedy nebo mluvené slovo. Kódování vymyslel roku 1840 Alfred Vail, asistent vynálezce Samuela Morse. Pomocí tohoto kódu bylo uskutečněno první telegrafické spojení mezi Washingtonem a Baltimorem.

Cílem této úlohy bylo zpracovat program, který by překládal věty napsané v Morseově abecedě na standartní text a naopak. Krom standartních písmen A-Z program převádí také čísla 0-9 a některé speciální znaky “.,?!;”. Jednou z nejpodstatnějších parametrů pro správné čtení slov v Morseově abecedě je jejich správné oddělení jednoho znaku od druhého pomocí mezery a následně oddělení jednotlivých slov pomocí „/”.

Popis kódu

Stejně jako předchozí kód tak se skládá z jedné třídy a uživatelského rozhraní pro komunikaci s programem. Program zapouzdřený ve třídě *MorseTranslator* jako taková se skládá ze 4 funkcí, které zpracovávají zadané vstupy.

První inicializační konstruktor `__init__`, který definuje atribut dictionary. Jedná se o slovník, který přiřazuje k jednotlivým znakům latinky, číslům a speciálním znakům, znaky Morseovy abecedy. Dále je zde vytvořen atribut *to_reverse_text*, který slouží k „obrácení“ slovníku pro opačný překlad z Morseovy abecedy na standartní text.

První metoda *to_morse* provádí kódování textu. Nejprve je vstupní text převeden na velká písmena pomocí funkce *upper*, aby byly jednotlivé znaky slovníku kompatibilní. Následně metoda projde celý soubor znak po znaku a pokud daný znak existuje přidá jeho znak do výsledného seznamu. Metoda je také opatřena ochranou před neznámými znaky, které čistě přeskočí.

```

Funkce To_Morse(text):
  Vstup: text (řetězec)
  Výstup: morse_kód (řetězec)

  1. Převeď 'text' na velká písmena (Upper Case)
  2. Inicializuj prázdný seznam 'výsledek'
  3. PRO KAŽDÝ 'znak' v 'text':
      POKUD 'znak' existuje v 'slovníku':
          Najdi 'kód' odpovídající 'znaku'
          Přidej 'kód' do seznamu 'výsledek'
      JINAK:
          Ignoruj znak
  4. Spoj prvky seznamu 'výsledek' pomocí mezery
  5. Vrať spojený řetězec

```

Obr. 3 – Algoritmus převodu z textu na Morseovu abecedu

Druhá metoda *to_text* zajišťuje dekódování v opačném směru a to na standardní text. Jednotlivé znaky jsou oddělovány mezerou a slova pomocí „/“. Tento znak „/“ jsem vložil také do slovníku což mi umožnilo zjednodušit kód a stačí tak pouhá mezera pro oddělení znaků. Následně jako u předchozí metody jsou jednotlivé znaky spojovány pomocí mezery.

```

Funkce To_Text(morse_kód):
  Vstup: morse_kód (řetězec kódů oddělených mezerou)
  Výstup: text (řetězec)

  1. Rozděl 'morse_kód' podle separátoru " " (mezera) do seznamu 'kódy'
  2. Inicializuj prázdný seznam 'výsledek'
  3. PRO KAŽDÝ 'kód' v seznamu 'kódy':
      POKUD 'kód' existuje v 'inverzním slovníku':
          Najdi 'znak' odpovídající 'kódu'
          Přidej 'znak' do seznamu 'výsledek'
  4. Spoj prvky seznamu 'výsledek' (bez oddělovače)
  5. Vrať spojený řetězec

```

Obr. 4 – Algoritmus převodu z Morseovy abecedy na text

Poslední metodou je *process_file* soubory jsou načítány kontextovým manažerem *with open*, který zajišťuje bezpečné uzavření souboru po dokončení práce. Součástí této metody je také ošetření výjimek pomocí *try-except*, které ochrání program před spadnutím při nenalezení souboru nebo neznámé chybě. Součástí této metody je také „přivolání“ příslušné metody pro zpracování souboru na základě vstupu uživatele, které je také ošetřeno ochranou proti zadání neplatné hodnoty.

Samotný běh programu zajišťuje blok *if name.....*, které pomocí cyklu *while* zajišťuje nekonečný běh programu dokud uživatel nezadá „end“ místo názvu souboru a tím je cyklus narušen a program se tak ukončí. Uživatel tedy hned po spuštění programu zadává název vstupního souboru (je mu doporučeno zadat celou cestu k souboru pro eliminaci chyb). Jako další uživatel vepíše název souboru, který bude vytvořen a do nějž se zapíše překlad. Jako poslední uživatel volí typ převodu a to zadáním 1 nebo 2. Využil jsem cyklu *while*, aby program nezpracoval pouze jeden soubor a následně už uživatel již nic nemohl vkládat.

Data a problematika zpracování

Jako vstup je využíván soubor .txt, který obsahuje znaky v Morseově abecedě oddělené mezerami a slova oddělená „/“ nebo standardní text. Hlavním omezením současného řešení je absence převodu znaků s diakritikou jako „Š, Č, Ř, Á...“, které nejsou součástí základní Morseovy abecedy. Dalším omezením je možnost chybovosti vstupního zapsání Morseovy abecedy, které nemusí být zapsán korektně dle pravidel toho, jak je program napsán a tím pádem by docházelo ke spojování slov nebo ke ztrátě informací spojením některých znaků, které tím pádem nebudou obsaženy ve slovníku.

Zdroje

- WIKIPEDIA (2026a): Kochova křivka
https://cs.wikipedia.org/wiki/Kochova_k%C5%99ivka (cit. 4.2. 2026)
- WIKIPEDIA (2026b): Morseova abeceda
https://cs.wikipedia.org/wiki/Morseova_abeceda (cit. 4.2. 2026)

Přílohy

- Správná vstupní data pro překlad z Morseovy abecedy do textu
- Správná vstupní data pro překlad z textu do Morseovy abecedy
- Kód pro tvorbu Kochovy vložky: kochova_vlocka.py
- Kód pro překlad Morseovy abecedy do textu a zpět: prekladac_morseovy_abecedy.py